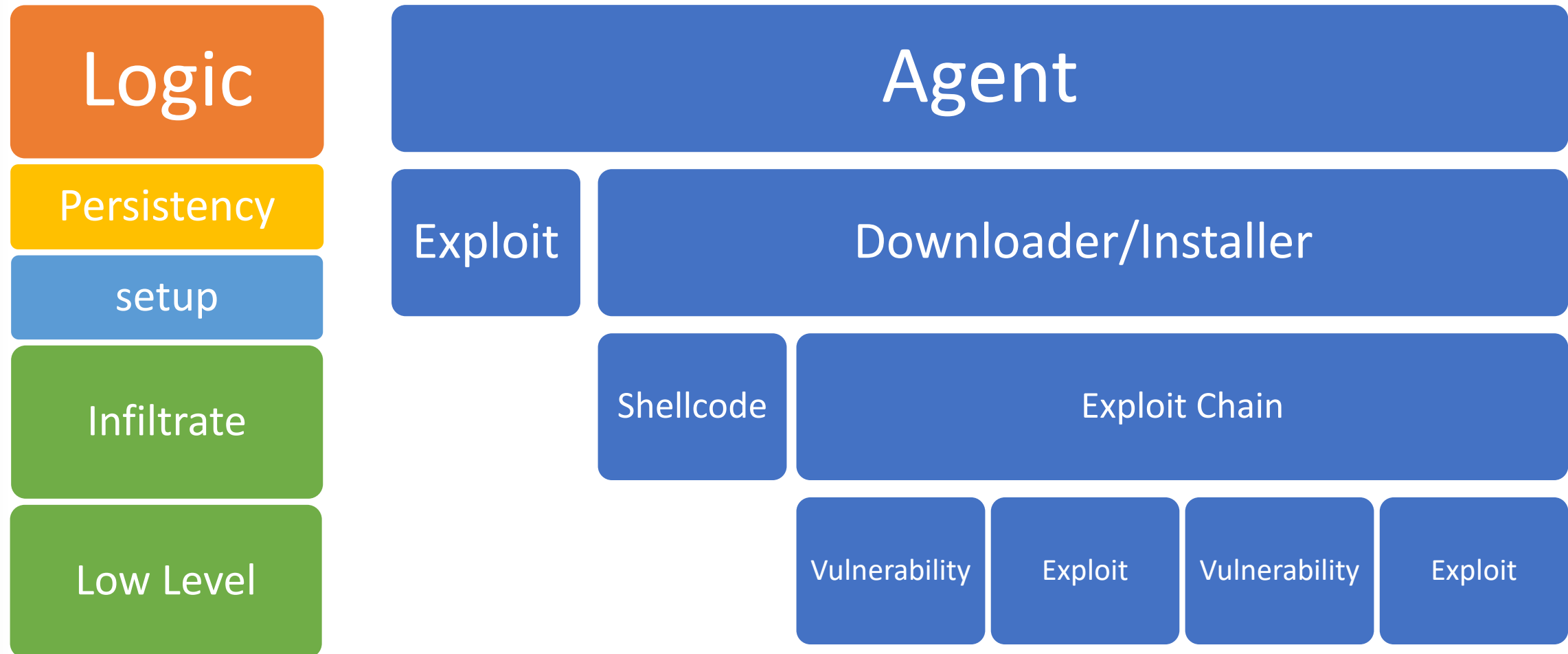


Advanced Exploitation

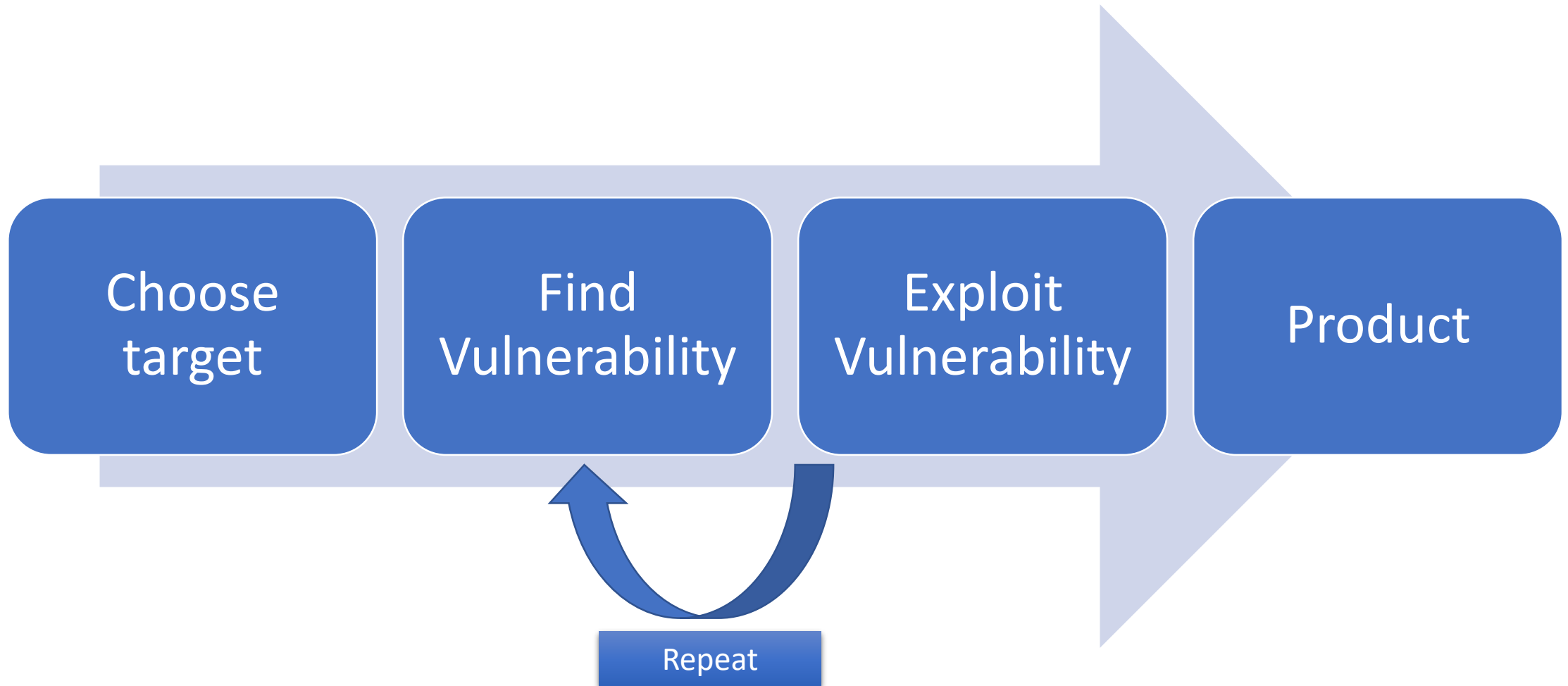
Or: Exploit Productization



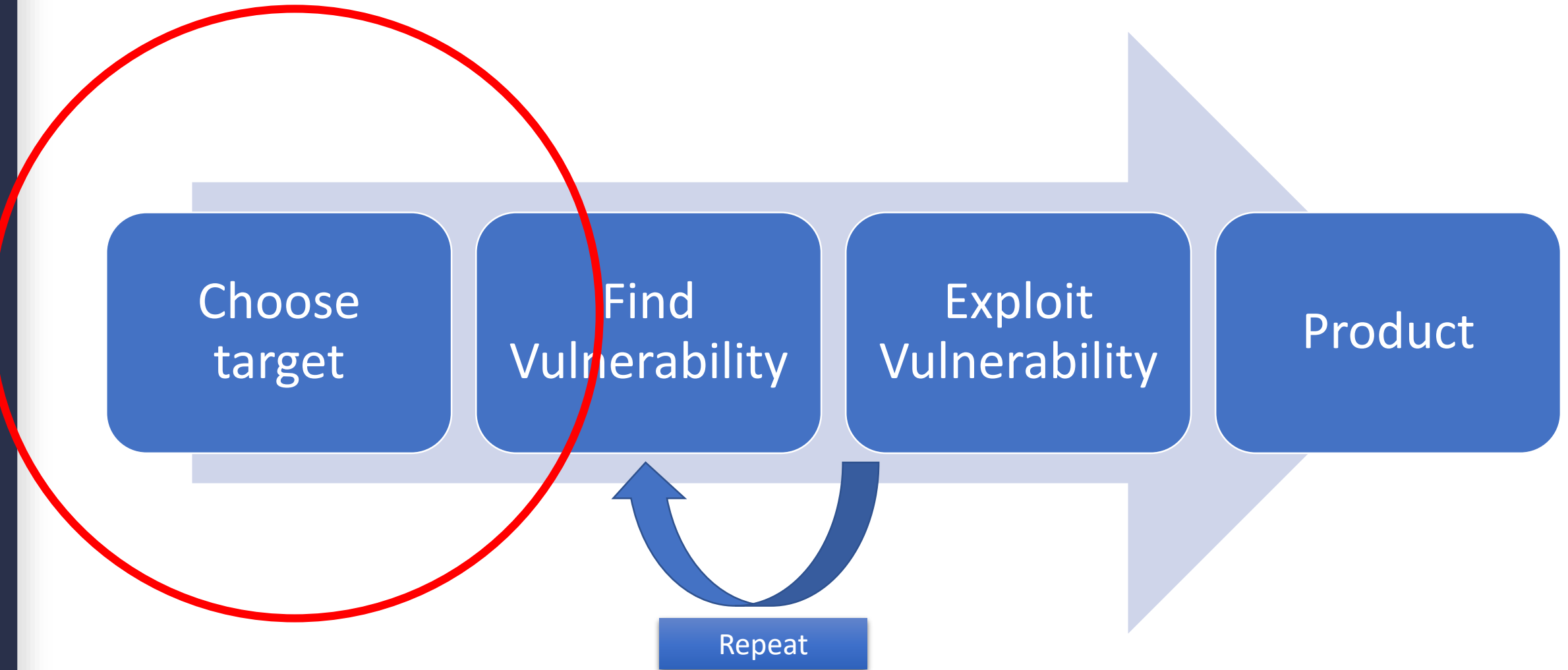
The Product



The Process

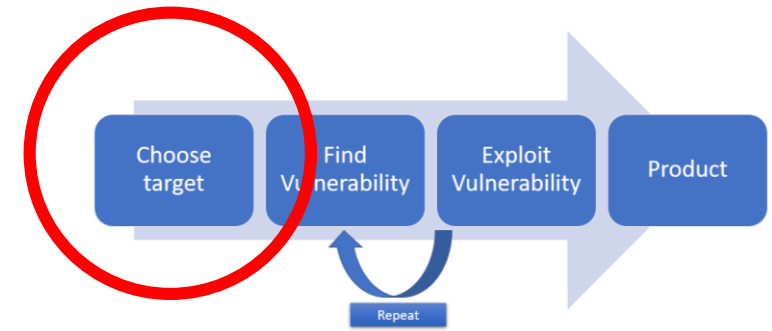


The Process



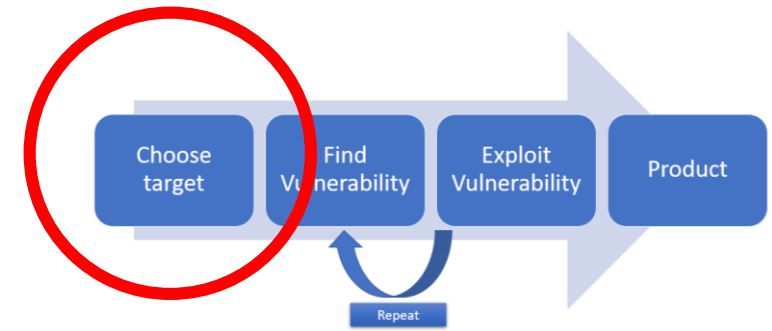
Choose Target

- Start by Getting Our bearings
- Map “Bug finding” criteria for each product
- Understand product “Meta-Data”
- How Big is the product?
- **How big is the attack surface?**
- **How Mature is the code?**



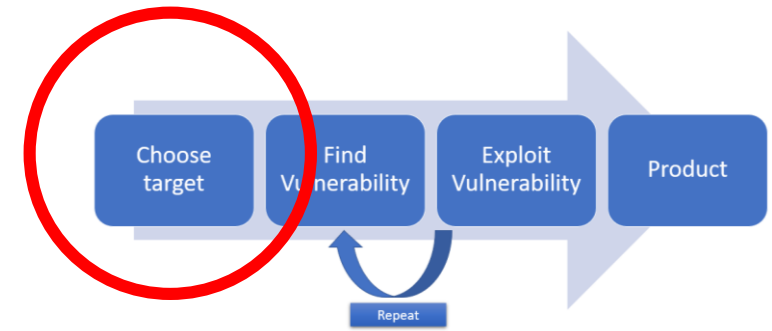
Choose Target

- How much code churn is there?
- Any open source?
- Any third party libraries?
- How well is the code maintained?
- **Does the vendor have a record of security?**



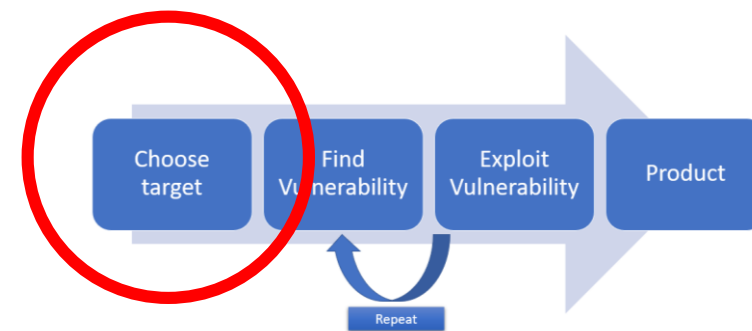
Choose Target

- Any shared code with other targets?
- Is the code compiled with mitigations?
 - Stack canaries
 - ASLR
 - Etc..
- Is the code compiled with symbols
- Are there prior RE efforts which can serve as documentation



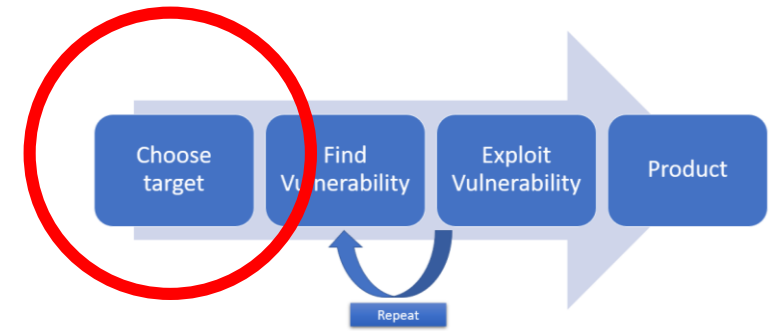
Choose Target

- **Does product receive regular updates?**
- **Were there many vulnerabilities in the past in the code?**
 - If there were in recent years – **there will be more**
 - But the 0-days might be short-lived
- Are many people looking at the code?
- Do we need a special setup to work with the device?



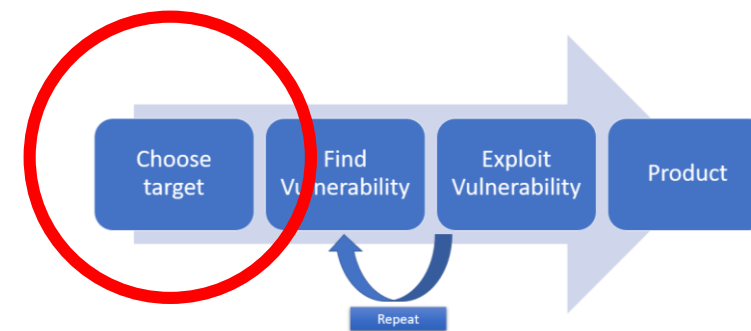
Choose Target

- How popular is the target among audience?
- What other bugs do we have in the chain?
- Project constraints
 - Can we tackle 2 targets at the same time
 - Do we need it fast? Do we need it robust?
 - Does it need to last long?
 - User interaction or operational constraints
 - How “Noisy” can we be?
 - **What is the attack scenario/vector?**

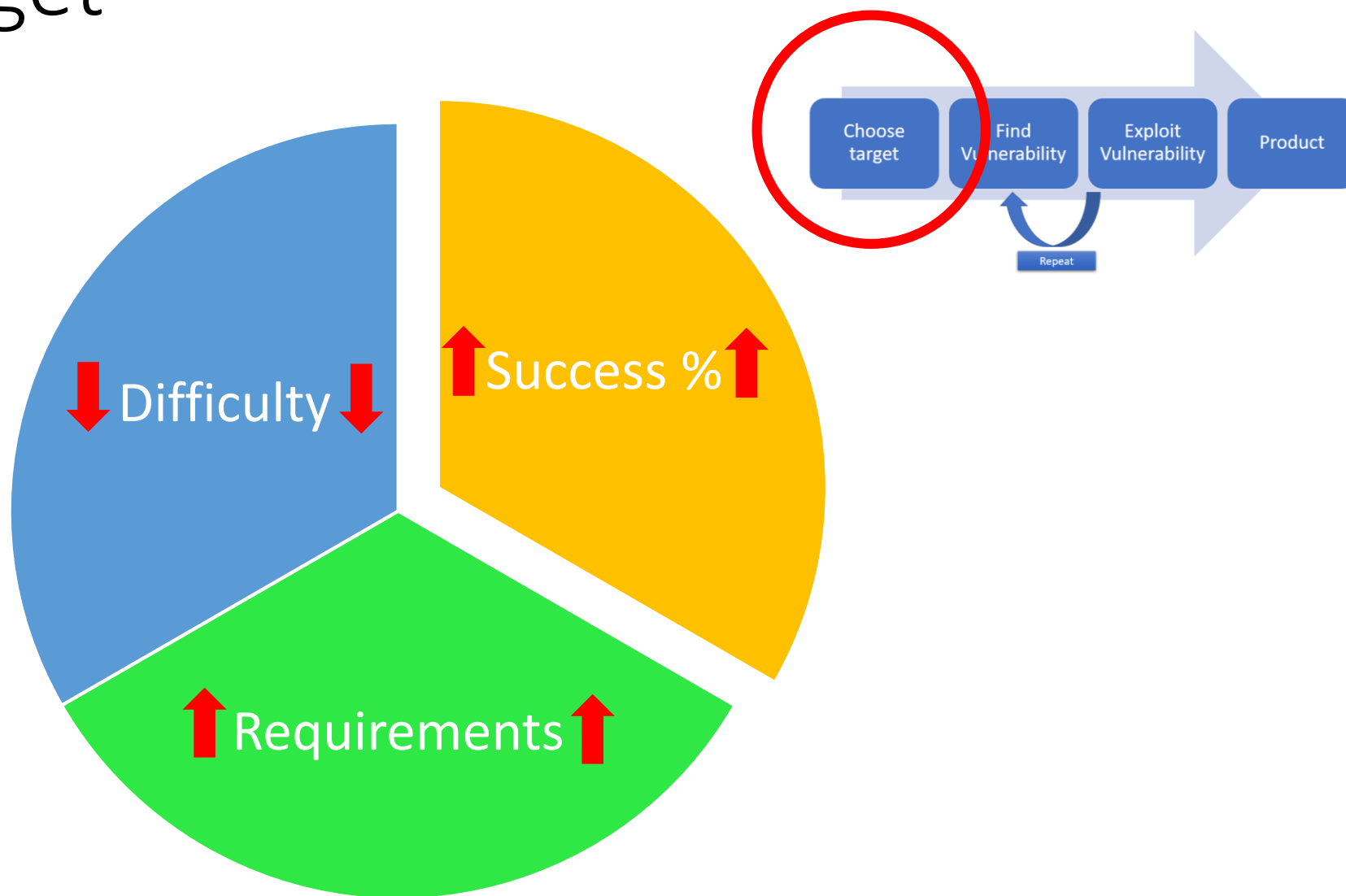


Choose Target

Feature/Target	notepad	Chrome	Microsoft Word	Skype	Bluetooth (Linux Bluez)
Code Quality	Unknown High	High	Medium	Unknown	Unknown
Code Churn	Low	high	high	medium	medium
Attack Surface	None	High	High	Medium	Medium
User-interaction	-	No	Yes	Maybe	No
Update frequency	Low	high	Medium	Medium	low
Attack Vector	-	MITM	Phishing	Phishing?	Proximity

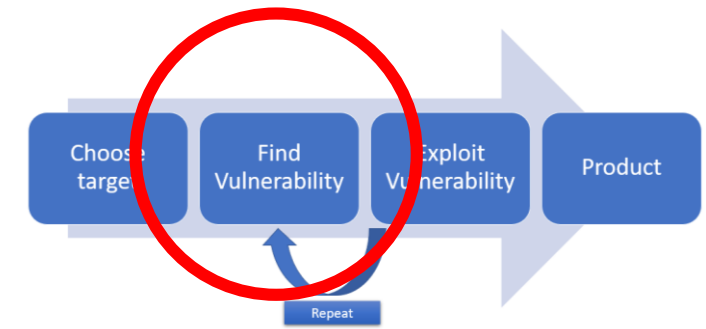


Choose Target



Find Vulnerability

- Depending on the Selected target and the project requirements
- Acquire Target
- Decide on Strategy for vulnerability hunting
- Search for vulnerability



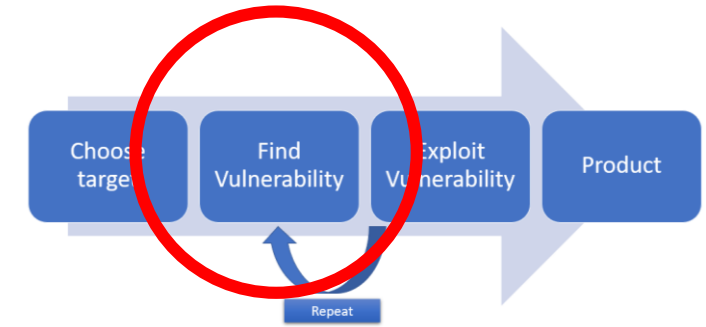
Acquire Target

- Before we can start working with a target we need to put our hands on it.
 - Meta-data
 - Documentation
 - Actual device
 - Firmware
 - Debugging capability
 - Source Code
- Sometimes we Can only get some of the info cost-effectively
 - Sometimes we make tradeoffs. Make a target decision based on meta-data



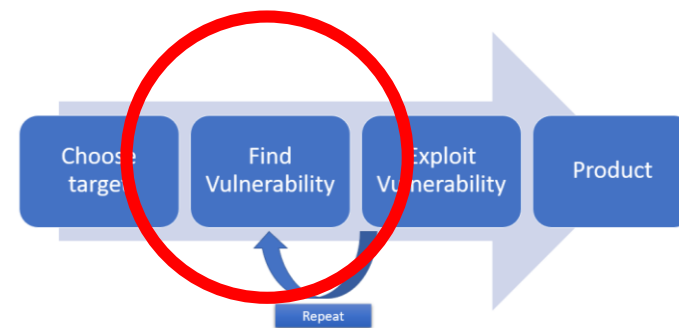
Acquire Target

- Sourcing Firmware is often an important stage
- We can get FW by reading the flash → Requires a lab
- Or Through UART shell
- Or JTAG
- Or update package
 - Will sometimes be encrypted or obfuscated
 - Available on website, update server or through vendor



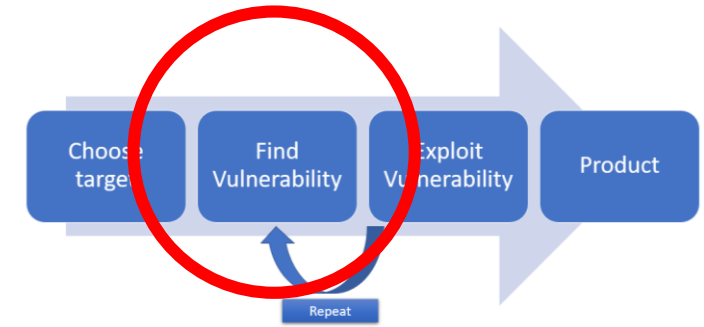
When to Use 1-days

- If the target doesn't update
 - This often makes sense
- If project requirements favor fast over long-term
- If we already have some 0-days and we want to close a chain
- If we have resources for exploitation but not for bug-hunt



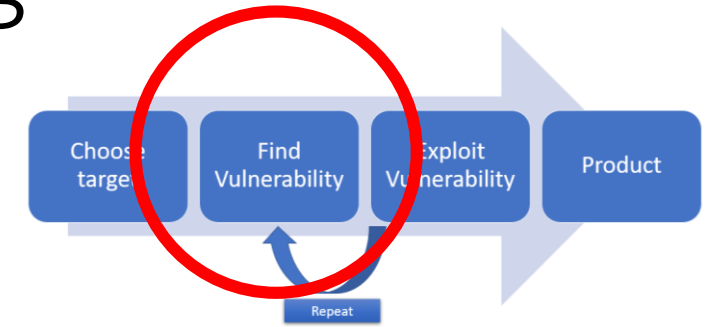
When to Fuzzing

- If we believe there are “Low hanging fruit”
- If attack surface is very large
- If the target is largely unexplored
- if we have capacity to fuzz **and** search manually
- If target is closed source and reversing is complex
 - Or if target is completely unavailable – this is not recommended



When to Code Review/Reversing

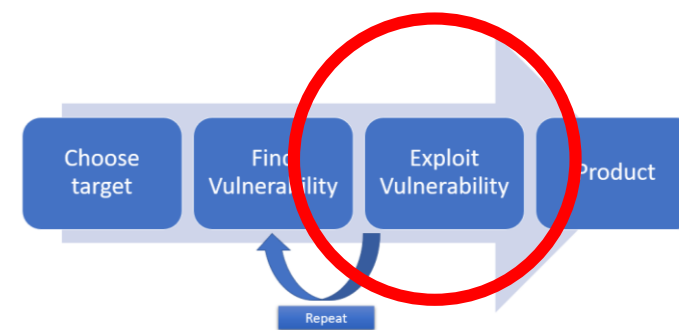
- In any other case
- This is the default
- As complementary to fuzzing
 - And to generate test cases or add coverage



You Have a vulnerability!

- Now we need to determine exploitability
- **Search around the bug for similar bugs or variations**
- Try to understand if exploitable
- Try to exploit
 - This will take time
 - And **can fail**





New vuln



Exploit

You Have a vulnerability!

- Exploitation for every target has a learning curve
- Exploitation is not always possible
- Sometimes you will need more vulnerabilities
- Rule of Thumb:
it will take you just as long to find the next vulnerability as it did the last



You Have an Exploit!

- You have an exploit – Great!
- In many cases you will need your exploit to work on several targets
- In many cases you will not be able to differentiate these targets remotely



You Have an Exploit!

- Multi-Target exploit
- Sometimes you can tailor a target to work on several versions at the same time
 - Rop that uses similarities for example
- Sometimes you will need find a “fingerprinting” method
 - Passive fingerprinting
 - Active fingerprinting
 - Infoleak / infoleak reuse

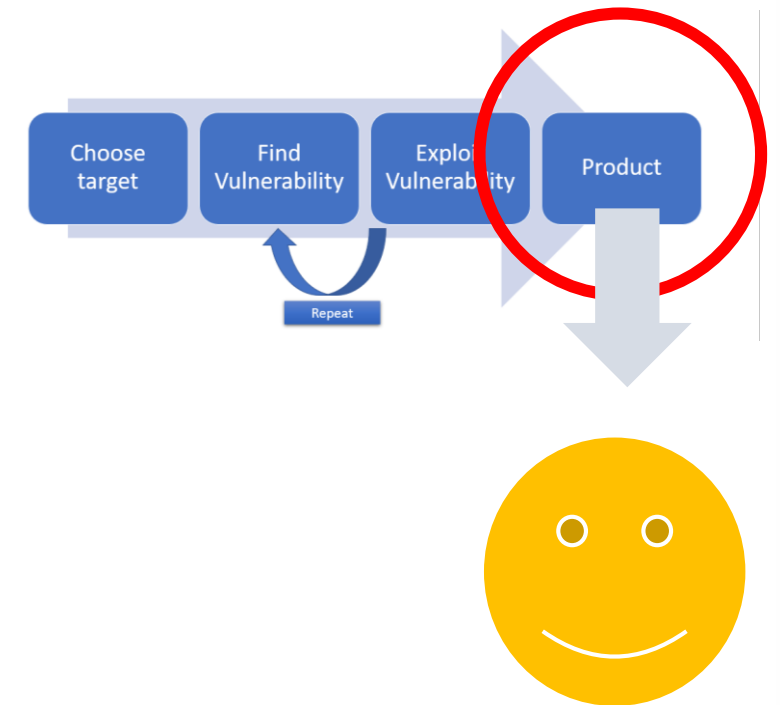


You Have an Exploit!

- you have a multi-target exploit: Multi-Great!
- In many cases you will need several exploits
 - If there are mitigations
 - If it is a modern complex OS
- RCE, PE, Persistency, Infoleak, infoleak, infolek

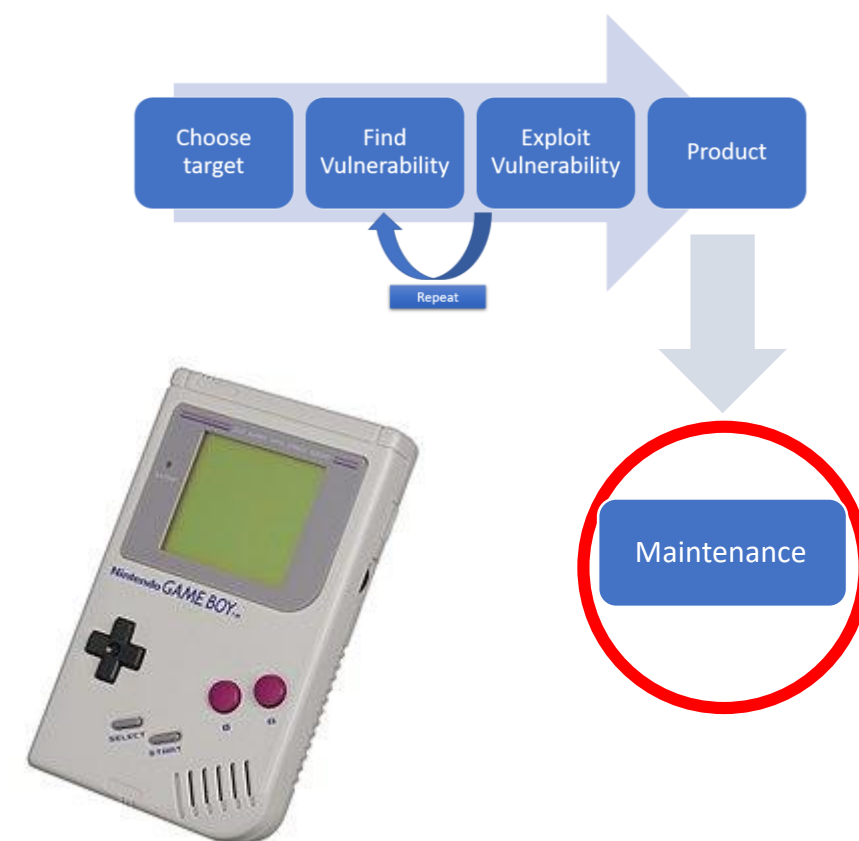


You Have an Exploit Chain!



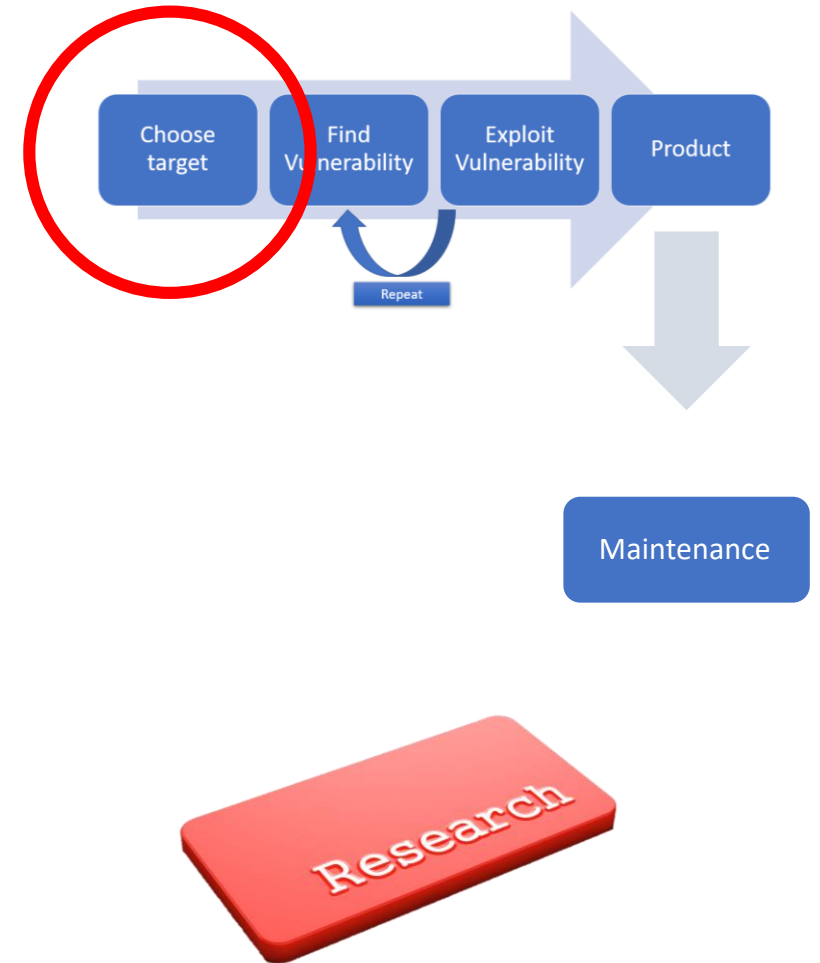
Maintenance

- Exploits need to be cared for
- New Software versions
- New mitigations
- Vulnerabilities close
 - Need to follow announcements
 - And **check if they were closed properly**
- Market Changes
 - Anyone need my brand new exploit for the **GAMEBOY?**



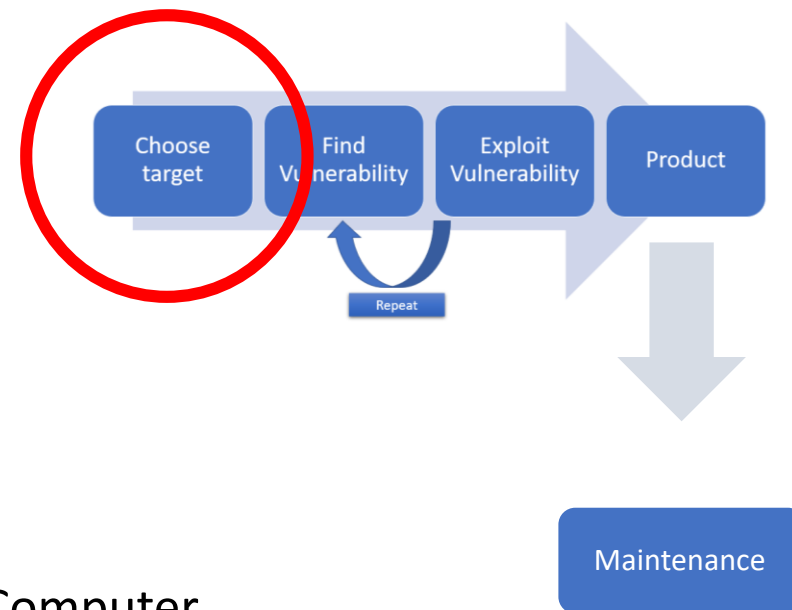
Exploit development process

- Exploit development process depends on the target
- It's Research
- We risk ourselves by trying to draw a timeline
- But we'll try to get some broad figures..



Exploit development process

- Target Selection includes sourcing targets and Code
- Can take 1 day to ~3 months
 - Depends on project requirements
 - And availability of targets
- Example: find a vulnerability in libtiff – 1 day
- Example find a vulnerability in Syrian Tank Engine management Computer
 - Source tank
 - Take pictures in Russian tank
 - Take apart computer
 - Extract firmware
 - Enumerate attack surfaces
 - Etc..
- Usually not many people



Exploit development process

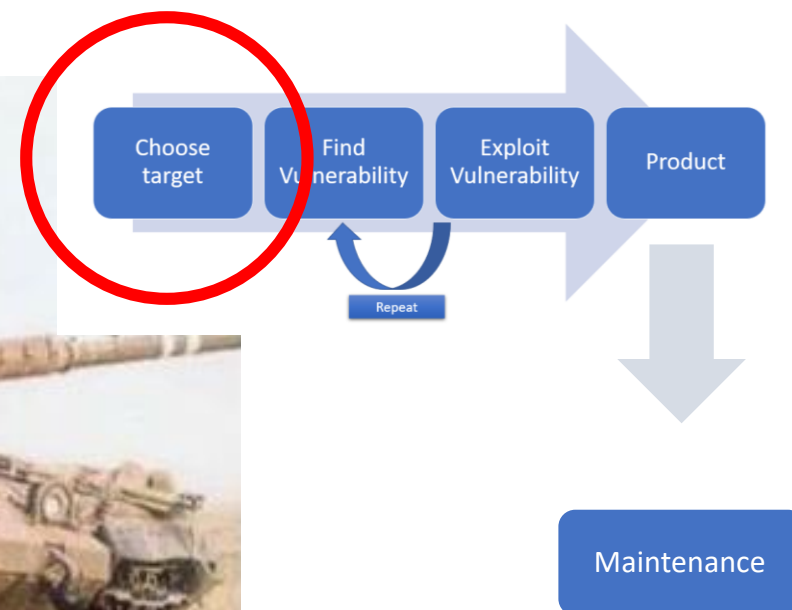
- Target Select

- Can take 1 d
 - Depends
 - And avail

- Example: find

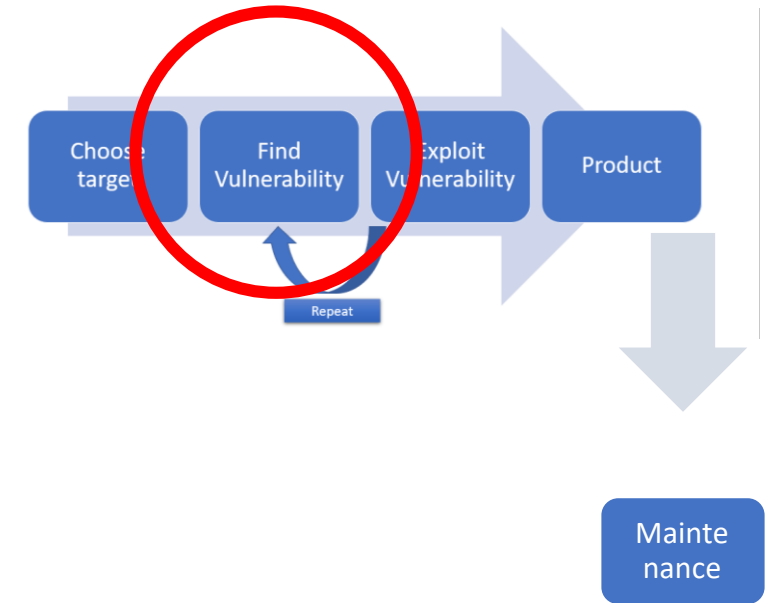
- Example find
 - Source ta
 - Take pictu
 - Take apar
 - Extract fir
 - Enumerat
 - Etc..

- Usually not many people



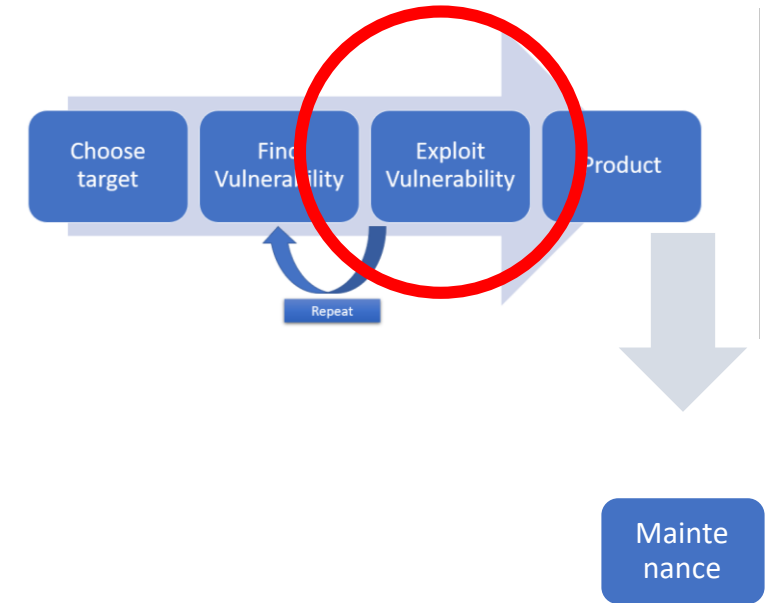
Exploit development process

- Finding a vulnerability depends on the target
- Easy target – 1 day – 4 weeks
 - For example: Random Chinese camera
 - For example: Well-known 1-day
- Very hard target – 1 year
 - For example: Apache Server
 - Requires Domain-specific expertise
- Can be split between several people
 - Depending on size of target



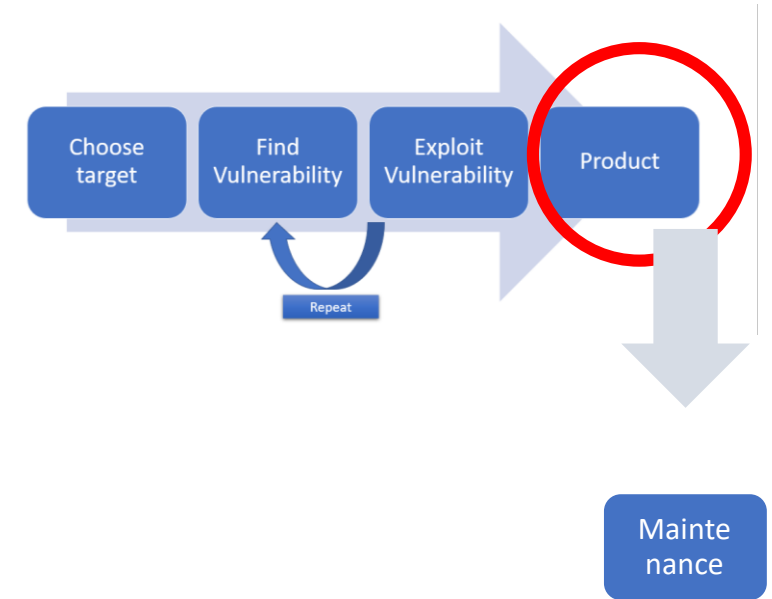
Exploit development process

- Exploitation takes approximately 2-6 months
- Of course depends on target and expertise
- And quality of the vulnerability



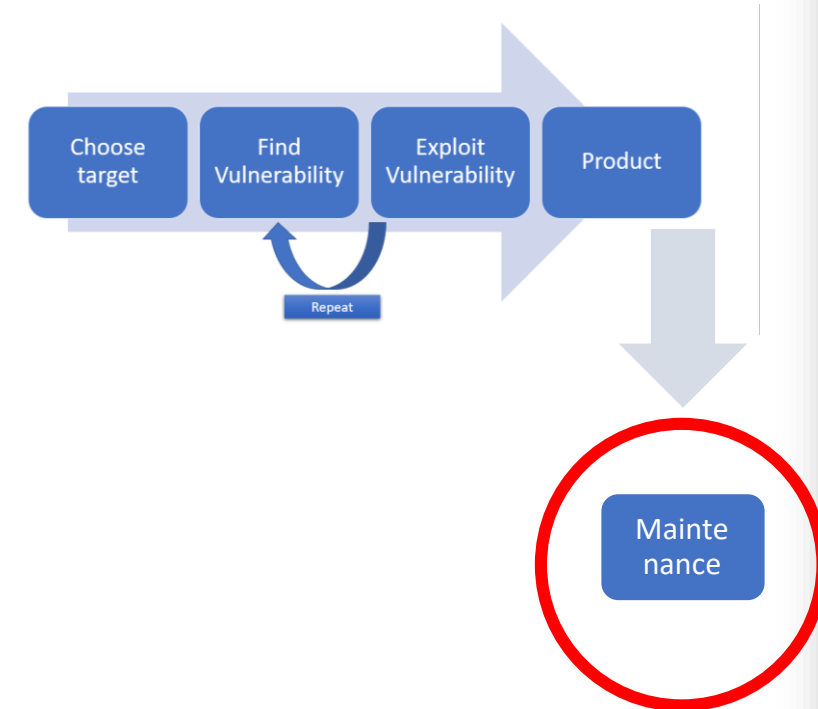
Exploit development process

- Includes Quality assurance, integration, docume
- Productization can take 1-4 months
- Depending on requirements

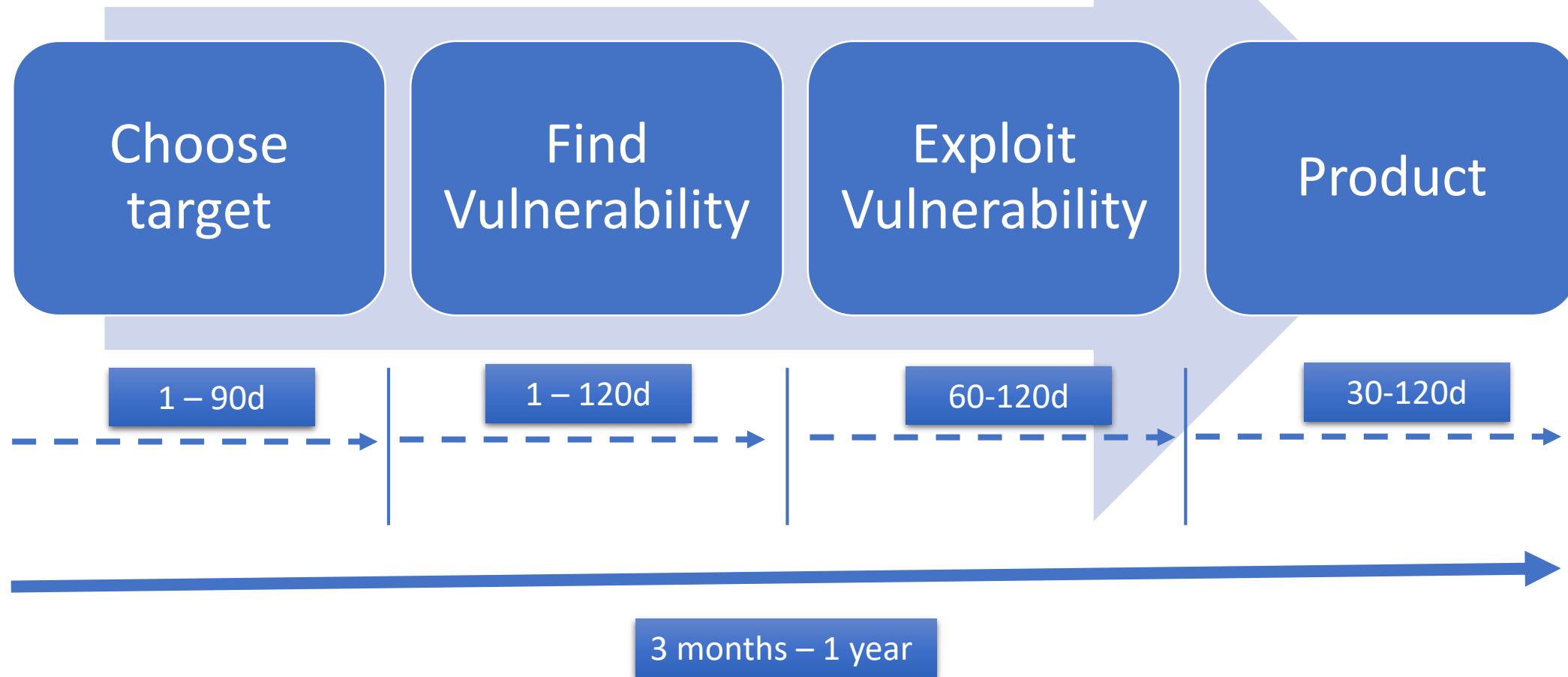


Exploit development process

- Maintenance is ongoing
- Requires approximately ½ position
 - Sometimes only requires someone on call
 - Sometimes someone full-time
- Depends on:
 - Update frequency
 - How important the project is



Exploit development process



Let's Practice

